Depends_on

Готовность и границы зависимости

Занятие 17.10.2026. Пишу от своего лица; практику выполнял с подсказками, оформление - с помощью ИИ. Показанный результат запуска отделяю от самостоятельного понимания, которое ещё нужно закреплять.

1. Зачем я добавил условие

В существующем проекте lab-compose-pair сервис web использует training-site:v2, proxy - nginx:1.30.5-alpine. Я продолжил работу после простого HTTP healthcheck: теперь нужно было связать запуск proxy с успешной проверкой web.

Запущенный процесс ещё не обязательно готов отвечать на запросы. Короткая запись зависимости сообщает Compose порядок запуска, но не требует успешного healthcheck:

```
depends_on:
  - web
```

В выводе compose config эта запись разворачивается в condition: service_started. Поэтому искать буквальное service_started в исходном коротком YAML не требуется.

2. Развёрнутая запись

По объяснению наставника я заменил зависимость внутри proxy:

```
depends_on:
  web:
    condition: service_healthy
```

- depends_on задаёт зависимость сервиса, внутри которого расположен блок.
- web - имя ожидаемого сервиса из services.
- condition задаёт условие: service_started ожидает запуска, service_healthy - успешного здоровья по healthcheck.
- Блок находится в proxy, потому что ждёт proxy. Сам healthcheck остаётся в web.

Готовность определяется конкретной проверкой. Наш wget обращается к http://127.0.0.1/ внутри web. Это локальная HTTP-загрузка, а не проверка всего приложения, DNS web или маршрута через proxy.

3. Как я наблюдал новый запуск

Наставник предложил один цикл stop и up для новой цели: увидеть ожидание при запуске proxy. Уже работающий proxy не демонстрирует условие собственного старта. Было объяснено, что сайт учебного проекта кратковременно станет недоступен, stop сохраняет контейнеры, а up может пересоздать изменившийся сервис, сохранив смонтированные данные.

На финальном скриншоте показан up -d и его результат. Выполнение stop и успешный финальный config на этом снимке отдельно не видны; я не приписываю им дополнительное доказательство.

Ошибки и исправления

4. Сайт отвечает, но ожидание ещё не настроено

На первом скриншоте 08:18 proxy уже был Up, web имел health: starting, а запрос через порт 8138 вернул HTTP 200 и Image version 2. Это подтвердило ответ сайта, но не ожидание здоровья при запуске.

Свежий compose config в 08:20 показал condition: service_started. Поэтому proxy мог запуститься, не дожидаясь успешной проверки. У web также обнаружилась ошибка аргумента wget: вместо -T было отдельное T. Следующий ps показал web unhealthy и работающий proxy. Наставник указал обе ошибки по присланному выводу.

5. Два разных дефиса

В развёрнутом YAML было:

```
- T
```

А требовалось:

```
- "-T"
```

Первый дефис обозначает элемент списка YAML. Дефис внутри кавычек является частью аргумента программы. Без него T не задаёт таймаут wget. Команда проверки может завершаться ошибкой, хотя основной веб-сервер продолжает работать. Точный текст ошибки из журнала healthcheck в этом опыте отдельно не получен.

6. Опечатка в условии

После изменения списка на развёрнутую зависимость compose config сообщил, что condition должен иметь одно из допустимых значений. Команды stop и ps тоже завершились на проверке этого YAML. Их действие в той попытке не выполнено: сначала Compose проверяет конфигурацию.

Следующий снимок исходного файла показал service_healty. Наставник указал пропущенную h после t. Я исправил значение на service_healthy. Отступы и имя параметра нужно отличать от точного допустимого значения: корректно читаемый YAML всё равно может не соответствовать схеме Compose.

Исправления аргумента, формы зависимости и написания значения сделаны с помощью. Самостоятельную диагностику этих ошибок я не заявляю.

Подтверждённый результат

7. Что показано в 08:32

- Вывод up -d: web Healthy, рядом 5.7s; proxy Started, рядом 0.2s.
- Вывод ps: web Up 20 seconds (healthy), создан 20 seconds ago; proxy Up 14 seconds, создан 14 hours ago.
- У proxy опубликован порт 127.0.0.1:8138 на порт 80 контейнера.
- Запрос с VM вернул HTTP/1.1 200 OK и тело <h1>Image version 2</h1>.

```
curl -i --max-time 3 http://127.0.0.1:8138/
```

Это подтверждает успешный запуск с ожиданием здоровья и работавший путь VM -> proxy -> web в момент запроса. Возраст proxy показывает, что новый запуск не обязательно создаёт новый контейнер.

8. Почему я ждал около шести секунд

Я спросил, нормально ли долгое развёртывание. Наставник объяснил: interval: 5s планирует первую проверку примерно через пять секунд после старта, а Compose ждёт healthy. Время около 5.7s согласуется с этим ожиданием. Разница Up у web и proxy около шести секунд также согласуется с порядком запуска.

Эти цифры не измеряют длительность инициализации Nginx или загрузки страницы. Точное разложение времени по этапам не измерялось; видимые длительности прогресса не являются точными временными метками запуска.

9. Что я сначала понял неверно

На вопрос о позднем unhealthy я ответил, что proxy остановится, потому что мы прописали это в коде. Наставник объяснил: condition: service_healthy задаёт ожидание перед запуском зависимого сервиса. Поздний unhealthy сам по себе не останавливает и не перезапускает уже работающий proxy и не перезапускает web. При этом запросы через работающий proxy могут завершаться ошибками или таймаутами.

После объяснения я сообщил: «а понял понял». Это подтверждает получение объяснения; самостоятельного ответа или применения после уточнения пока нет. Аналогичный блок worker -> api мной не представлен.

10. Мой итог

Практический блок выполнен с помощью, финальные healthy и HTTP 200/v2 показаны. Самостоятельность не повышаю, весь модуль Docker не закрыт. Следующая тема - DNS и localhost, путь запроса VM -> proxy -> web и назначение logs, inspect, exec на работающем проекте. Новый отказ повторять сейчас не требуется; поведение при позднем unhealthy закрепим коротким переносом на другой пример.

Источники: Порядок запуска Compose, depends_on, HEALTHCHECK, compose up, compose stop.